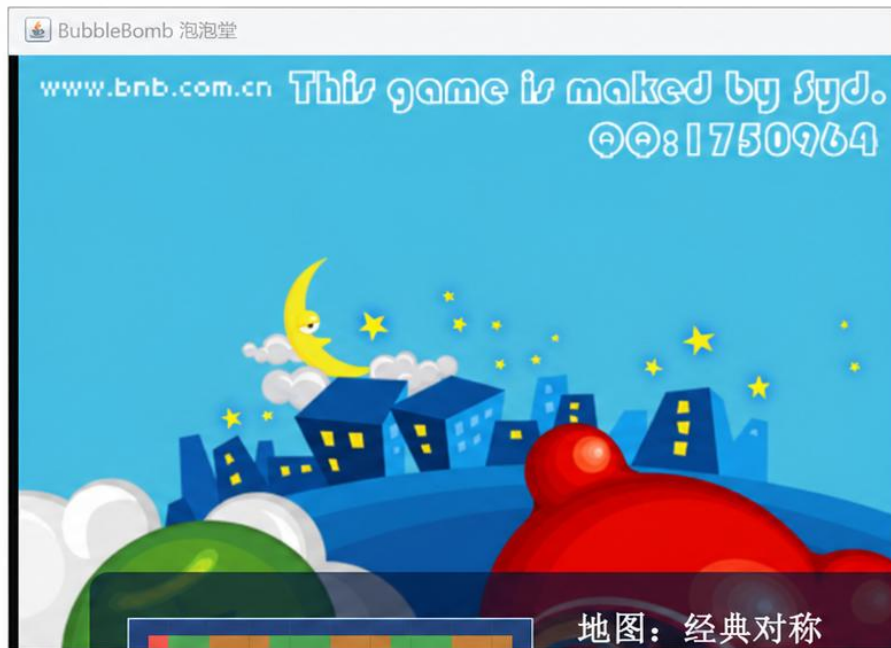


BubbleBomb 泡泡堂



Java Swing 本地双人对战游戏

陈丽文 · 张晴
初级软件实作课程答辩

项目背景与开发目标

① 以经典泡泡堂玩法为原型

② 完成可独立运行的 JavaSE 桌面游戏

需求 → 设计 → 实现 → 验证

③ 将面向对象、事件驱动和模块化设计
用于真实项目

需求分析与游戏规则

- ① 本地双人同时对战
- ② 放置泡泡、破坏箱子、获取道具
- ③ 比赛采用先胜 3 回合制



初始界面与地图选择

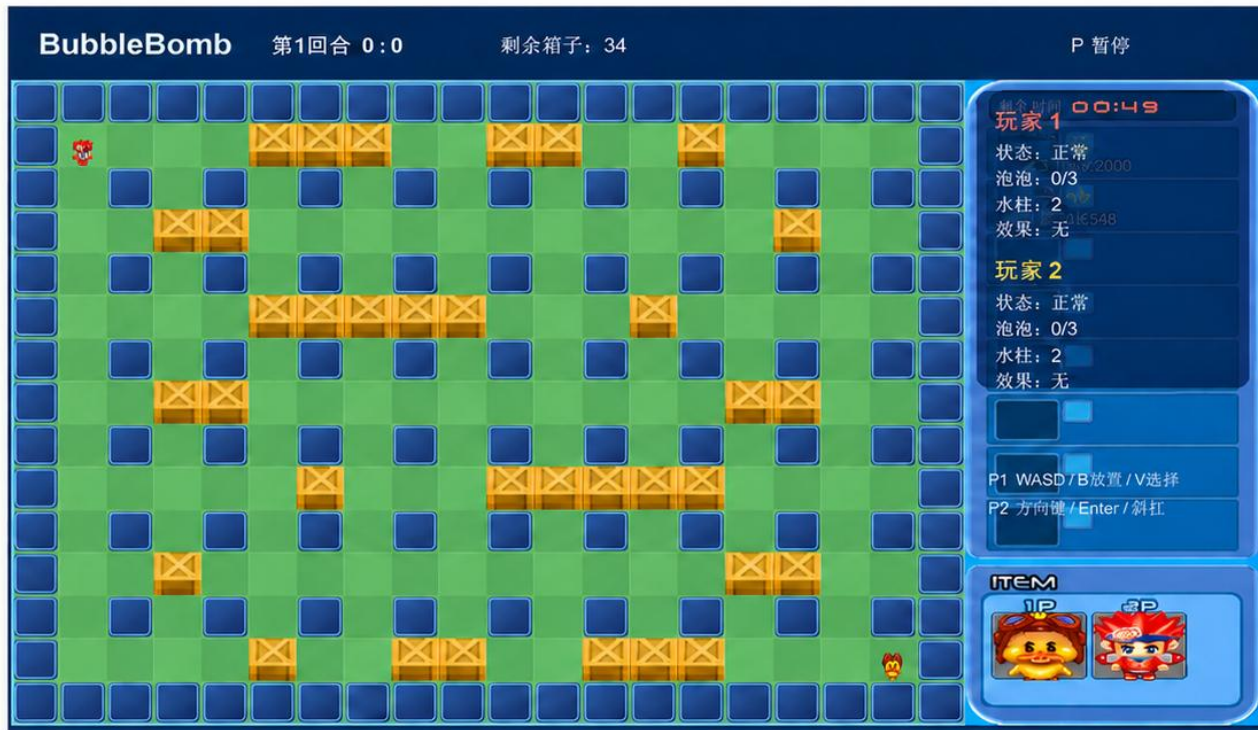


方向键选择地图与箱子密度

H 查看游戏说明, I 查看道具说明

4 类地图满足不同对战节奏

实际游戏运行



20 × 15 格对战地图

状态栏 实时 显示比分与角色能力

Swing 完成地图、角色与元素绘制

功能总览

4 类地图 · 3 档箱子密度

9 种道具 · 多种异常与强化效果

积分赛 · 决胜倒计时 · 对称缩圈

道具说明

 泡泡：增加泡泡设置数目（获得两个增加一个泡泡）

 蓝药瓶：增加泡泡爆炸威力。

 紫药瓶：消除反向行走效应。

 紫鬼头：玩家获得产生反向行走效应。

 红鬼头：玩家获得后对手产生反向行走效应。

 遥控器：玩家获得后对手停止飞行。

 魔水雷：玩家获得对手被泡泡包围。

 超人卡：玩家获得增加一次抵御魔水雷效应。

几点说明：

图标的效果和泡泡堂游戏有的不一样，大多都是作者自己想出来的。

玩家可以根据自己想法修改脚本达到其他效果。

by:Strike ©©:175096590

泡泡堂

Nothing ventured, nothing gained.
www.bnk.com.cn



六阶段开发计划

1 需求与框架

2 基础功能

3 核心玩法

4 游戏流程

5 整合测试

6 提交答辩



均已完成

工程目录与分层结构



```
1 BubbleBombGame/
2 | src/
3 |   App.java
4 |   com/bubblebomb/
5 |     config/      GameConfig.java
6 |     controller/  GameListener.java  InputState.java
7 |     game/        GameStart.java  GameState.java  GameMap.java
8 |     show/        GameFrame.java  GamePanel.java
9 |     manager/     ElementManager.java  ResourceManager.java
10 |    element/     Player.java  Bubble.java  WaterBlast.java
11 |      prop/      Prop.java  PropType.java
12 | resources/
13 |   config/       game.properties  images.properties
14 |   maps/         level_01.map  level_02.map  level_03.map
15 |   images/       23 PNG resources
```

controller 负责输入

game / show 负责流程与界面

element / manager 负责元素和资源



游戏主循环与状态机

16 ms 定时更新 · 7 种界面状态

- ① Swing Timer 每 16 ms 驱动更新与重绘;
- ② GameState 统一管理标题、运行、暂停与结算;
- ③ GamePanel 根据状态更新元素并执行胜负判断。

输入 → 更新 → 判断 → 重绘

```
1 // GameState.java - seven UI states
2 public enum GameState {
3     TITLE, HELP, ITEM_HELP, RUNNING,
4     PAUSED, ROUND_OVER, GAME_OVER
5 }
6
7 // GamePanel.java - 16 ms update loop
8 private GameState state = GameState.TITLE;
9
10 private void tick() {
11     if (state == GameState.RUNNING) {
12         elements.update(input);
13         String result = elements.getWinnerText();
14         if (result != null) {
15             finishRound(elements.getWinnerId(),
16             } else {
17                 updateShowdown();
18             }
19         }
20     }
21     input.endTick();
22     repaint();
23 }
```

玩家输入、移动与碰撞

① InputState 记录持续按键与单次按键

② Player 将水平、垂直输入转换为位移

③ canMove 在更新坐标前完成碰撞判断

输入 → 位移 → 碰撞 → 更新

```
1 // InputState.java
2 public void press(int keyCode) {
3     if (downKeys.add(keyCode)) {
4         pressedKeys.add(keyCode);
5     }
6 }
7
8 // Player.java
9 int horizontal = (input.isDown(rightKey) ? 1 : 0)
10    - (input.isDown(leftKey) ? 1 : 0);
11 int vertical = (input.isDown(downKey) ? 1 : 0)
12    - (input.isDown(upKey) ? 1 : 0);
13
14 boolean moved = false;
15 if (dx != 0 && manager.canMove(this, getX() + dx, getY())) {
16     setX(getX() + dx);
17     moved = true;
18 }
19 if (dy != 0 && manager.canMove(this, getX(), getY() + dy)) {
20     setY(getY() + dy);
21     moved = true;
22 }
```

地图、元素与资源管理

```
1 // ElementManager.java
2 public void resetLevel(GameMap map, int boxDensity) {
3     clear();
4     int tile = config.tileSize();
5     List<String> lines = getBaseMap(map);
6     for (int row = 0; row < lines.size(); row++) {
7         String line = lines.get(row);
8         for (int column = 0; column < line.length(); column++) {
9             char cell = line.charAt(column);
10             int x = column * tile;
11             int y = row * tile;
12             if (cell == '#') add(new Wall(x, y, tile));
13         }
14     }
15 }
16
17 // ResourceManager.java
18 public BufferedImage getImage(String key) {
19     BufferedImage cached = imageCache.get(key);
20     if (cached != null) return cached;
21     BufferedImage image = readImage(imagePaths.getProperty(key, key));
22     imageCache.put(key, image);
23     return image;
24 }
```

● GameMap 维护地图名称、路径与特点

● ElementManager 统一创建、更新和清理元素

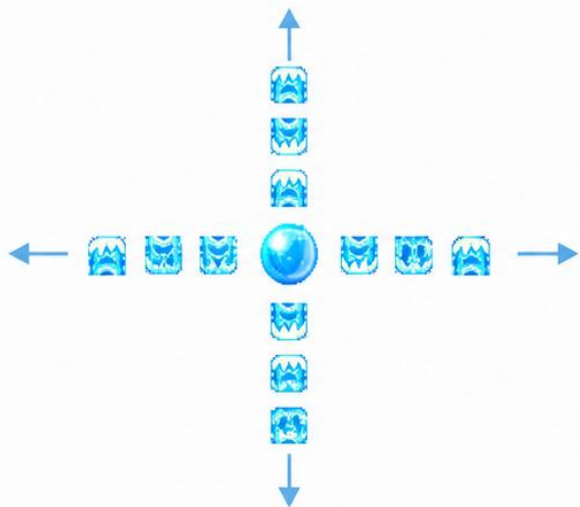
● ResourceManager 使用缓存避免重复读取图片

泡泡、水柱与连锁爆炸

Bubble 引信结束后进入触发状态

水柱向四个方向逐格传播并遇阻停止

命中其他泡泡后继续触发连锁爆炸



```
1 // Bubble.java
2 public void update(ElementManager manager, InputState input) {
3     if (--fuseTicks <= 0) {
4         triggered = true;
5     }
6 }
7
8 // ElementManager.java
9 private void processTriggeredBubbles() {
10     boolean found;
11     do {
12         found = false;
13         List<ElementObj> snapshot =
14             new ArrayList<ElementObj>(elements.get(ElementType.BUBBLE));
15         for (ElementObj element : snapshot) {
16             Bubble bubble = (Bubble) element;
17             if (bubble.isLive() && bubble.isTriggered()) {
18                 explode(bubble);
19                 found = true;
20             }
21         }
22     } while (found);
23 }
24
25 if (otherBubble instanceof Bubble && otherBubble.isLive()) {
26     ((Bubble) otherBubble).trigger();
27     break;
28 }
```

箱子、道具与特殊地形

```
1 // Prop.java
2 public void applyTo(Player player) {
3     switch (propType) {
4         case BUBBLE_UP: player.increaseBubbleCapacity(); break;
5         case RANGE_UP: player.increaseBlastRange(); break;
6         case ANTIDOTE: player.clearDebuffs(); break;
7         case REMOTE: player.grantRemote(); break;
8         case WATER_MINE: player.grantWaterMine(); break;
9         case DEMON_RED: player.applySlow(); break;
10        case DEMON_PURPLE: player.applyReverse(); break;
11        case GOLD_CARD: player.increaseSpeed(); break;
12        case DIAMOND_CARD: player.grantShield(); break;
13        default: break;
14    }
15    setlive(false);
16 }
17
18 // Terrain.java
19 if (terrainType == TerrainType.SPEED) {
20     g.fillRect(x + 2, y + 2, getWidth() - 4, getHeight() - 4);
21 } else {
22     g.fillOval(x + 2, y + 5, getWidth() - 4, getHeight() - 10);
23 }
```

道具说明

-  泡泡: 增加泡泡设置数目 (获得两个增加一个泡泡)
-  蓝药瓶: 增加泡泡爆炸威力。
-  紫药瓶: 消除反向行走效应。
-  紫鬼头: 玩家获得产生反向行走效应。
-  红鬼头: 玩家获得后对手产生反向行走效应。
-  遥控器: 玩家获得后对手停止遥控。
-  魔水雷: 玩家获得对手被泡泡包围。
-  超人卡: 玩家获得增加一次抵御魔水雷效应。

① 箱子破坏后按规则随机掉落道具

② PropType 统一定义 9 种道具效果

③ 加速地面和泥地提供延时速度效果

积分赛与决胜缩圈

先胜 3 回合 = 比赛结束

120 秒 — 箱子清空后开启决胜

60 秒 — 开始对称缩圈

30 秒 — 显示红色警告

```
1 // GamePanel.java
2 private void updateShowdown() {
3     if (!showdownActive && elements.getBoxCount() == 0) {
4         showdownActive = true;
5         showdownTicks = Math.max(1,
6             config.showdownSeconds() * 1000 / config.tickMillis());
7         elements.prepareSuddenDeath();
8     }
9     showdownTicks--;
10    if (showdownTicks <= config.shrinkStartSeconds()
11        * 1000 / config.tickMillis()) {
12        elements.advanceSuddenDeathWall();
13    }
14 }
15
16 private void finishRound(int winnerId, String result) {
17     if (winnerId == 1 || winnerId == 2) {
18         scores[winnerId - 1]++;
19     }
20     if (winnerId > 0 && scores[winnerId - 1] >= config.matchWins()) {
21         state = GameState.GAME_OVER;
22     } else {
23         state = GameState.ROUND_OVER;
24     }
25 }
```

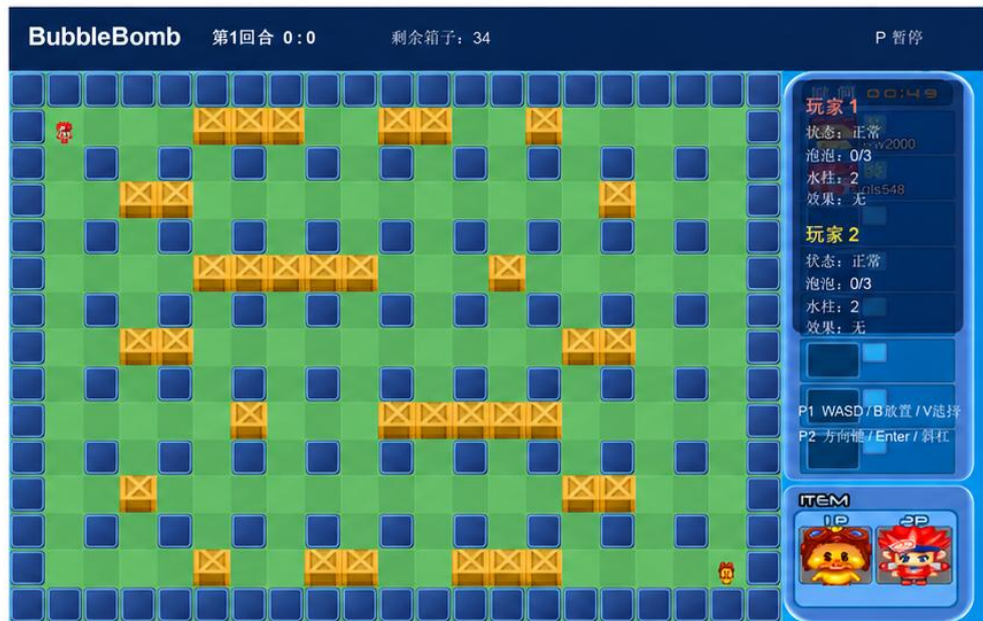
项目整合与测试结果

JDK 11 及以上，不依赖第三方库

24 个 Java 文件 · 约 2127 行代码 ·

23 个图片资源

启动、输入、碰撞、爆炸、道具、
暂停、回合和缩圈流程均已检查



功能 运行正常

小组分工与项目总结

陈丽文

需求分析 · 工程框架

玩家移动与碰撞 · 泡泡与水柱 · 连锁爆炸

状态管理 · 胜负判断 · 代码整合

运行说明 · 实训报告

张晴

图片资源 · 初始界面 · 游戏窗口

固定地图 · 箱子与道具 · 特殊地形

积分赛 · 决胜缩圈 · 界面优化

功能测试 · PPT · 答辩视频

AI 辅助：PPT 结构、视觉排版、素材整理和答辩文字优化

答辩视频：使用真人录音，不使用 AI 配音

六阶段开发已完成，项目功能运行正常

谢谢 / Q&A